TRIUMPH CHURCH PRODUCTION RIG

Recovery Guide

Rebuilding the rig from nothing: the Pi, Companion, its scripts, its backups and its credentials

Read off the live rig October 27, 2026 at 10:20 AM · Companion 5.0.7+9763-stable-cec2f88e6f

Contents

1. Read this first
2. How the rig fits together
3. Backups
4. Rebuild the Pi from nothing
5. When one thing breaks
6. Credentials
7. Which tool builds which page
8. Keeping this manual true

Read this first

This book is for rebuilding the rig when the person who built it is not around. It assumes you can use a terminal and follow steps exactly; it does not assume you know Companion.

What you need before you start

	Where it is
A computer on the **AV network** (10.23.0.x)	Cabled into the production switch, or the church VPN. Church Wi-Fi alone does not reach it
1Password, vault "Production"	Every password this book names. Entries are written **1Password › Entry name**
This repo	**github.com/samuelbailey123/companion-icons** (public). The Pi's scripts are in `pi/`, the tools that build each page in `tools/`
The Service Health board's repo	**github.com/Decima-Labs/service-health** (private: ask a Decima Labs admin for access)
The latest Companion backup	See *Backups* below
The three PDFs	This book, the Operator Guide and the Reference, in the shared Google Drive folder *Production rig manual* and in `manual/` in the repo

The one rule: nothing changes on the rig while a service is live or recording (Sundays from 10:00, and any event being recorded). Read-only checks are fine at any time.

How the rig fits together

Everything is driven by **Bitfocus Companion**, running on a **Raspberry Pi** in the rack. The Stream Decks plug into the Pi by USB. Companion talks to each device over the AV network: the switcher, the router, the cameras, ProPresenter, the sound desk, the lighting desk and the LED wall. A few things Companion cannot do itself are done by small scripts on the Pi that Companion runs: projector and PA power, the camera readouts, and the walk-in music on the PP1 iMac.

Next to Companion, the same Pi runs the **Service Health board** (http://production/), which checks every device and shows one verdict.

Part	What it is	Where
The Pi	Raspberry Pi 5, 8 GB, 256 GB SD card, Debian 13 (CompanionPi image), hostname `CompanionPi`	Rack; **10.23.0.242** by DHCP
Companion	Bitfocus Companion 5 (5.0.7 on 2026-10-27), service `companion.service`, runs as user `companion`	Web UI **http://10.23.0.242:8000**. Program in `/opt/companion`, settings in `/etc/companion/config.yaml`, configuration in `/home/companion/.config`
The scripts	Python and shell, run by Companion's "run shell command" action	`/home/samuelbailey/Desktop/AV_Power_scripts` (backed up in `pi/AV_Power_scripts`)
The board	Node app, service `service-health.service`, runs as `samuelbailey`	`/home/samuelbailey/service-health`; http://production/ and :8080
Nightly backup	`companion-backup.timer` at 03:17, writes to a USB drive labelled `storage`	`/mnt/storage/companion-backups` (see *Backups*: **the drive is missing**)
Stream Decks	Elgato Stream Deck + XL and Stream Deck +	USB into the Pi

Companion updates itself. The Pi has gone 5.0.3 → 5.0.4 → 5.0.7 unattended. An update restarts Companion and the deck goes dark for a minute or two. If an update ever breaks a page, the module versions in the Reference (*Connections*) are the ones known to work.

Backups

There are three, and each one restores a different thing.

Backup	What it holds	Where	How fresh
Companion full export	Every page, key, trigger, variable, connection (with its passwords) and image	Google Drive, *Production rig manual* / *Companion backups* (private)	Whenever someone runs the export. Take one before every change
Nightly config archive	Companion's whole configuration folder	USB drive labelled `storage` on the Pi, `/mnt/storage/companion-backups`, 30 days kept	Nightly at 03:17, **when the drive is plugged in**
This repo	The Pi's scripts (`pi/`, passwords taken out), the tools that build each page, this manual	GitHub	Each commit

> **Found on 2026-10-27: the nightly backup is not running.** No drive is plugged into the Pi, so the 03:17 job fails every night ("Dependency failed": it needs `/mnt/storage`). Until a USB drive formatted ext4 with the label `storage` is plugged into the Pi, the only Companion backups are the ones someone takes by hand.

Take a Companion export now (from any computer on the AV network):

```
curl -s http://10.23.0.242:8000/int/export/full | gunzip > companion-$(date +%F).json
```

or, in a checkout of this repo, `node tools/rig.js export companion-$(date +%F).json`, or in Companion's web UI, the **Import / Export** page, **Export**, full configuration. **The file contains device passwords in plain text.** Keep it in the private Drive folder or 1Password, never in the repo.

Refresh the scripts backup after any script on the Pi changes: `node tools/pi-backup.js`, then commit `pi/`. It copies them over SSH and swaps each password for its 1Password entry name.

Rebuild the Pi from nothing

Use this when the Pi is dead, its SD card has failed, or Companion will not start and cannot be fixed. Allow two hours. Do it on a weekday.

1. Hardware

A Raspberry Pi 5 (8 GB is what is there now), its official power supply, a 64 GB or larger SD card, and an Ethernet cable to the production switch. Plug the two Stream Decks into its USB ports.

2. Install Companion

Install **CompanionPi**, Bitfocus's Raspberry Pi build of Companion: follow the current instructions at **bitfocus.io/companion** (download, or the CompanionPi install script on top of Raspberry Pi OS Lite 64-bit). When it asks, call the machine `CompanionPi`. Install the same major version the Reference's cover page names (Companion 5).

Create the user the scripts expect, because Companion's keys call them by full path:

```
sudo adduser samuelbailey            # any password; record it as 1Password › Companion Pi login
sudo usermod -aG sudo,users samuelbailey
sudo usermod -aG users companion     # Companion must be able to read the tracks SSH key
```

In `/etc/companion/config.yaml`, keep `adminPort: 8000` and set `enableShellCommandSupport: true`: the Power page, PTZ readouts and walk-in music all depend on Companion being allowed to run shell commands. Then `sudo systemctl restart companion`.

3. Give it its address

The Pi gets **10.23.0.242** from the church's UniFi gateway (*TC-BMT UDM Pro*, unifi.ui.com, 1Password › UniFi). A new Pi has a new network card, so update the DHCP reservation to its MAC address (Client Devices → the Pi → Fixed IP 10.23.0.242). Keep the DNS record **production** → **10.23.0.242** (Settings → Policy Table → DNS). The board, the bookmarks and this manual all assume that address.

4. Restore the scripts

From a checkout of this repo:

```
scp -r pi/AV_Power_scripts samuelbailey@10.23.0.242:Desktop/
```

Then on the Pi, put the passwords back (each file says which 1Password entry it needs):

File	What to put back	1Password entry
`projector_power.py`	`PASSWORD = "..."`	Projectors (Panasonic) web + PJLink
`speaker_power.py`	`PASSWORD = "..."`	PA power relay (DLI DIN)
`.ptz_web` (new file, `chmod 600`)	one line, `user:password`	PTZ cameras web login

The walk-in music script reaches the PP1 iMac over SSH with a key of its own. Make a new one:

```
cd ~/Desktop/AV_Power_scripts
ssh-keygen -t ed25519 -N '' -f .chrome_tracks_key -C companion-tracks
chgrp users .chrome_tracks_key && chmod 0640 .chrome_tracks_key
ssh-copy-id -i .chrome_tracks_key.pub tcmedia@10.23.0.204   # 1Password › PP1 iMac (tcmedia)
ssh -i .chrome_tracks_key -o UserKnownHostsFile=.chrome_tracks_known_hosts tcmedia@10.23.0.204
true
chgrp users .chrome_tracks_known_hosts && chmod 0660 .chrome_tracks_known_hosts
```

The header of `chrome_tracks.sh` explains why the key is group-readable rather than 0600, and what the iMac needs once: Remote Login on, the "allow sshd to control Google Chrome" prompt approved on its screen, and Chrome's *View → Developer → Allow JavaScript from Apple Events* ticked.

Check each one by hand before going on: `python3 projector_power.py state` and `python3 speaker_power.py state` print `on`, `off`, `mixed` or `unreachable`; `./chrome_tracks.sh state` prints the player's state.

In the repo	On the Pi
`pi/AV_Power_scripts/.coveragerc`	`/home/samuelbailey/Desktop/AV_Power_scripts/.coveragerc`
`pi/AV_Power_scripts/PROTOCOLS.md`	`/home/samuelbailey/Desktop/AV_Power_scripts/PROTOCOLS.md`
`pi/AV_Power_scripts/README.md`	`/home/samuelbailey/Desktop/AV_Power_scripts/README.md`
`pi/AV_Power_scripts/chrome_tracks.sh`	`/home/samuelbailey/Desktop/AV_Power_scripts/chrome_tracks.sh`
`pi/AV_Power_scripts/projector_power.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/projector_power.py`
`pi/AV_Power_scripts/ptz_look_10.23.0.181.json`	`/home/samuelbailey/Desktop/AV_Power_scripts/ptz_look_10.23.0.181.json`
`pi/AV_Power_scripts/ptz_look_10.23.0.196.json`	`/home/samuelbailey/Desktop/AV_Power_scripts/ptz_look_10.23.0.196.json`
`pi/AV_Power_scripts/ptz_params.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/ptz_params.py`
`pi/AV_Power_scripts/ptz_reference_params.json`	`/home/samuelbailey/Desktop/AV_Power_scripts/ptz_reference_params.json`
`pi/AV_Power_scripts/ptz_resave.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/ptz_resave.py`
`pi/AV_Power_scripts/ptz_state.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/ptz_state.py`
`pi/AV_Power_scripts/ptz_web.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/ptz_web.py`
`pi/AV_Power_scripts/speaker_power.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/speaker_power.py`
`pi/AV_Power_scripts/test_projector_power.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/test_projector_power.py`
`pi/AV_Power_scripts/test_speaker_power.py`	`/home/samuelbailey/Desktop/AV_Power_scripts/test_speaker_power.py`
`pi/README.md`	`(notes: stays in the repo)`
`pi/system/companion-backup.service`	`/etc/systemd/system/companion-backup.service`
`pi/system/companion-backup.sh`	`/usr/local/bin/companion-backup.sh`
`pi/system/companion-backup.timer`	`/etc/systemd/system/companion-backup.timer`

5. Restore Companion's configuration

Open http://10.23.0.242:8000, the **Import / Export** page, **Import**, choose the newest full export from the Drive folder, and import **everything**. Every page, trigger and connection comes back, passwords included.

With no export, but with the nightly archive: stop Companion (`sudo systemctl stop companion`), unpack the newest `companion-config-*.tar.gz` into `/home/companion/` as the `companion` user, and start it again.

With neither: each page can be rebuilt from this repo, one tool per page (table below), against a fresh Companion with its connections added by hand from the Reference's *Connections* chapter. That is a day's work, which is why the backups matter.

6. Put the nightly backup back

```
sudo cp pi/system/companion-backup.sh /usr/local/bin/ && sudo chmod 755 /usr/local/bin/companion-backup.sh
sudo cp pi/system/companion-backup.service pi/system/companion-backup.timer /etc/systemd/system/
echo 'LABEL=storage /mnt/storage ext4 defaults,nofail,x-systemd.device-timeout=10 0 2' | sudo tee -a /etc/fstab
sudo mkdir -p /mnt/storage && sudo mount /mnt/storage
sudo mkdir -p /mnt/storage/companion-backups && sudo chown companion /mnt/storage/companion-backups
sudo systemctl daemon-reload && sudo systemctl enable --now companion-backup.timer
sudo systemctl start companion-backup.service && ls /mnt/storage/companion-backups
```

The last line must list a fresh `companion-config-….tar.gz`.

7. Put the board back

Follow *Deploy to the Pi* in the Service Health repo's README: Node via asdf at the version in `.tool-versions`, the repo in `/home/samuelbailey/service-health`, `npm ci --omit=dev`, and its systemd unit. It also needs:

- `~/.config/service-health/pco.env` (mode 0600): the Planning Center token, 1Password › Planning Center token (pco-lint).
- The board's passphrase for schedule editing: `node ~/service-health/src/set-passphrase.ts`, 1Password › Service Health board passphrase.

8. Prove it

1. http://production/ shows every device green (on a weekday: *Off day*, and every device that is powered reads Up).
2. On the deck, every page's readouts are live: ATEM names, camera pan/tilt numbers, SQ7 DCA levels, the Dante page green.
3. Power page: switch the projectors on and off once.
4. Take a fresh Companion export and put it in the Drive folder.

When one thing breaks

Symptom	Likely cause	Fix
One page's readouts are blank, keys do nothing	The device's address changed (DHCP), or it is off	Find its new address (the board, or the UniFi client list), then in Companion **Connections**, edit that connection's host. Better: give the device a DHCP reservation in UniFi
Every page dead, deck dark	Companion stopped or is mid-update	`ssh samuelbailey@10.23.0.242`, `systemctl status companion`; `sudo systemctl restart companion` if it is not updating. Logs: `journalctl -u companion -n 200`
Power page does nothing	Script missing, or a projector/relay password changed	On the Pi: `python3 ~/Desktop/AV_Power_scripts/projector_power.py state`. "authentication failed" means the password in the script is wrong; `/tmp/av_power.log` has the history
PTZ readouts red	Camera off, address changed, or `ptz_web.py` cannot log in	`python3 ~/Desktop/AV_Power_scripts/ptz_state.py <camera address>`; check `.ptz_web`
Walk-in music keys do nothing	The iMac is asleep or off, or the SSH key was lost	`./chrome_tracks.sh state` on the Pi; re-do the key steps in *Restore the scripts*
Dante page red	A receiver or computer is off or unplugged	The red key names it. Dante Controller on a Mac on the AV network shows the same
Videohub keys stale	The router is found by Bonjour; its saved address is from another network	Companion **Connections** → **videohub**: set host to 10.23.0.21
Board down, deck fine	The board's service stopped	`systemctl status service-health`; `journalctl -u service-health -n 100`

Credentials

1Password, vault "Production". Nothing in this repo or these books contains a password; each is named here by its entry. Companion's own export does contain them, which is why exports live in the private Drive folder.

1Password entry	What it opens	Used by
Companion Pi login	SSH and sudo on the Pi (`samuelbailey@10.23.0.242`)	Rebuilds, script changes, the board's deploys
PP1 iMac (tcmedia)	The ProPresenter iMac's only account, Screen Sharing and SSH (10.23.0.204)	Walk-in music key, fixing ProPresenter
Projectors (Panasonic) web + PJLink	All three projectors' web, NTCONTROL and PJLink login	`projector_power.py`
PA power relay (DLI DIN)	The Digital Loggers relay that switches the PA (10.23.0.250)	`speaker_power.py`
PTZ cameras web login	Both FoMaKo cameras' web pages (10.23.0.181, .196)	`ptz_web.py` via `.ptz_web`
Planning Center token (pco-lint)	Planning Center API, read-only Personal Access Token	Companion `pco`, the board
Service Health board passphrase	Schedule and plan editing on the board	People editing Sunday from the board
UniFi	The church's gateway (*TC-BMT UDM Pro*), DHCP reservations and DNS	Address changes
ATEM stream key	The streaming platform key set in ATEM Software Control	The ATEM's stream
GitHub	`samuelbailey123/companion-icons`, `Decima-Labs/service-health`	Changing the rig's tools or the board

The connections in Companion that hold a credential, and the entry each belongs under:

Connection	Module	Fields	Entry
propresenter	renewedvision-propresenter-api	`pass`, `sdpass`	1Password › **Companion** › **propresenter**
grandma2	malighting-grandma2	`pass`	1Password › **Companion** › **grandma2**
pco	planningcenter-serviceslive	`applicationid`, `secretkey`	1Password › **Companion** › **pco**

> **Change the PA relay's password.** It is the same word as the PP1 iMac's account name, which this repo has to publish (the walk-in music script logs in as that account), so it is effectively public. Found 2026-10-27.

Anyone on the AV network can read Companion's configuration and log, passwords included: Companion's admin page has no login. Treat the AV network as trusted-only, and assume anything typed into a Companion connection is visible on it.

Which tool builds which page

Every page on the deck was built by a script in `tools/`, which reads a live export, builds the page and writes a bundle that `tools/rig.js` imports. Each script's header says why the page is the way it is; read it before changing that page.

Tool	What it builds
`tools/atem-page.js`	Rebuild the ATEM page as a proper switcher surface on the Stream Deck + XL.
`tools/atem-stream.js`	Add guarded Stream and Record keys to the ATEM page.
`tools/atem-tally.js`	Improve the ATEM page: preview tally, and a program/preview readout on the touchstrip.
`tools/dante-page.js`	The Dante page, a read-only SQ7 sub-page, and the SQ7 and Home keys that open it.
`tools/home-hub.js`	Turn linear page navigation into a home page with one folder per page.
`tools/home-page.js`	Home: the page you land on, showing what is happening without pressing anything.
`tools/lighting-page.js`	Rebuild the grandMA2 lighting page's knobs and touchstrip.
`tools/ma2-page.js`	MA2: an executor row across the top, and an icon on every key.
`tools/macro-icons.js`	Give every ATEM macro key on the rig the symbol the module never shipped with.
`tools/mics-page.js`	Mics: one key per channel, drawn by the receiver's own status display.
`tools/nav-buttons.js`	Replace Companion's built-in page up/down keys with styled buttons that name their
`tools/pi-backup.js`	Refresh `pi/`, the backup of the Pi's scripts, from the live Pi.
`tools/power-page.js`	Rebuild the Power page as two state-aware toggles.
`tools/pp1-countdown-trigger.js`	Fire the pre-service countdown at a fixed time, every Sunday.
`tools/pp1-countdown.js`	PP1: one key that fires the pre-service countdown.
`tools/pp1-freeze.js`	Put the Freeze key on PP1.
`tools/pp1-icons.js`	Give every PP1 button a real library icon, legible on the background it actually sits on.
`tools/pp1-page.js`	Show the ProPresenter timer on the PP1 page.
`tools/pp1-preservice-trigger.js`	Start the pre-service pre-roll at 10:00 every Sunday, and check that it started.
`tools/pp1-preservice.js`	PP1: one key that fires the pre-service pre-roll.
`tools/pp1-stage-layout.js`	PP1: one key that swaps the stage screen between its two service layouts.
`tools/pp1-timers.js`	PP1: two clocks on the strip, and a key per preset that fires one.
`tools/pp1-tracks-trigger.js`	Start the backing tracks at 10:00 Central every Sunday, and fade them out so they are silent at
`tools/pp1-tracks.js`	A Tracks folder on PP1: play/pause, previous and skip, a fade out, what is playing, and a volume
`tools/ptz-page.js`	Replace the Mics page with the PTZ camera page, and rebuild the folder row everywhere.
`tools/ptz2-page.js`	Emit the PTZ pages: a chooser, a run and setup page for each of the two cameras, and the
`tools/readable-labels.js`	Make button captions readable and CONSISTENT.
`tools/record-disk.js`	Make the Record keys show which disk the switcher will record to.
`tools/record-hyperdeck.js`	Move recording from the ATEM to the HyperDeck: the Record keys and the Countdown key.

Tool	What it builds
`tools/rig.js`	Talk to the live Companion over its admin API, the way its own web UI does.
`tools/shure-page.js`	Build the Shure wireless page, and the connections behind it.
`tools/sq-page.js`	SQ7: a DCA row across the top, each key showing its level and whether it is muted.
`tools/sunday-power-trigger.js`	Turn the projectors and the PA on at 08:05 Central every Sunday.
`tools/switch-page.js`	The Videohub page on the + XL, and VW repointed at the router's post-move ports.
`tools/system-page.js`	System: the Pi's health, spread across the deck instead of packed into four columns.
`tools/system-stats.js`	Build a System page watching the Raspberry Pi that Companion runs on, plus the trigger
`tools/vh-page.js`	VH: a destination row over a source row, the way a router is actually operated.
`tools/vh-routes.js`	Improve the VideoHub page: show which route is actually live.
`tools/vw-page.js`	Rename VH to VW, and give the LED wall a brightness encoder.
`tools/wire.js`	Generate per-page .companionconfig files with library icons wired onto buttons.
`tools/xl-relayout.js`	Rebuild every page for the Stream Deck + XL: folder row on top, everything else relocated.

To rebuild one page: `node tools/rig.js export live.json`, run the page's tool against `live.json` as its header says, then `node tools/rig.js import-page <bundle> <page number>`. Off-air only.

Keeping this manual true

The Reference is generated from the live rig; the Operator Guide and this book are written by hand, with their tables generated. After any change to the rig, on the AV network:

```
node tools/manual/collect.js     # reads the rig; writes .snapshot/ (holds passwords; never
committed)
node tools/manual/build.js       # writes manual/*.pdf
```

Then commit `manual/` and replace the PDFs in the Drive folder. The build refuses to write a book that contains any password from the export, or an Operator Guide whose Sunday timeline is missing a trigger.